

Ultimate Linked List Notes in Java

1. What is Linked List?

Linked List is a linear dynamic data structure.

Each node contains:

- Data
- Address of next node

Structure:

10 -> 20 -> 30 -> null

2. Node Class

```
class Node {  
  
    int val;  
    Node next;  
  
    Node(int val) {  
  
        this.val = val;  
        this.next = null;  
    }  
}
```

3. Basic Linked List Creation

```
class BasicLL {  
  
    Node head;  
  
    void create() {  
  
        Node a = new Node(10);
```

```
        Node b = new Node(20);
        Node c = new Node(30);

        head = a;

        a.next = b;
        b.next = c;
    }
}
```

BASIC OPERATIONS

4. Traversal / Print Linked List

```
void print() {
    Node temp = head;

    while(temp != null) {
        System.out.print(temp.val + " -> ");

        temp = temp.next;
    }

    System.out.println("null");
}
```

5. Recursive Print

```
void recursivePrint(Node head) {

    if(head == null)
        return;

    System.out.print(head.val + " ");
}
```

```
        recursivePrint(head.next);  
    }  
}
```

6. Reverse Print

```
void reversePrint(Node head) {  
  
    if(head == null)  
        return;  
  
    reversePrint(head.next);  
  
    System.out.print(head.val + " ");  
}
```

7. Search Element

```
boolean search(int key) {  
  
    Node temp = head;  
  
    while(temp != null) {  
  
        if(temp.val == key)  
            return true;  
  
        temp = temp.next;  
    }  
  
    return false;  
}
```

8. Update Element

```
void update(int oldVal, int newVal) {  
  
    Node temp = head;
```

```
while(temp != null) {  
    if(temp.val == oldVal) {  
        temp.val = newVal;  
        return;  
    }  
    temp = temp.next;  
}
```

9. Find Size / Length

```
int size() {  
    int count = 0;  
    Node temp = head;  
    while(temp != null) {  
        count++;  
        temp = temp.next;  
    }  
    return count;  
}
```

10. Check Empty List

```
boolean isEmpty() {  
    return head == null;  
}
```

INSERTION OPERATIONS

11. Add at Beginning

```
void addFirst(int val) {  
  
    Node temp = new Node(val);  
  
    temp.next = head;  
  
    head = temp;  
}
```

12. Add at End

```
void addLast(int val) {  
  
    Node temp = new Node(val);  
  
    if(head == null) {  
        head = temp;  
        return;  
    }  
  
    Node t = head;  
  
    while(t.next != null) {  
        t = t.next;  
    }  
  
    t.next = temp;  
}
```

13. Insert at Position

```
void insertAt(int pos, int val) {  
  
    if(pos == 1) {  
        addFirst(val);  
        return;  
    }  
  
    Node temp = new Node(val);  
  
    Node t = head;  
  
    for(int i = 1; i < pos - 1; i++) {  
        t = t.next;  
    }  
  
    temp.next = t.next;  
  
    t.next = temp;  
}
```

14. Insert in Middle

```
void insertMiddle(int val) {  
  
    Node newNode = new Node(val);  
  
    if(head == null) {  
        head = newNode;  
        return;  
    }  
  
    Node slow = head;  
    Node fast = head;  
    Node prev = null;  
  
    while(fast != null && fast.next != null) {
```

```
        prev = slow;

        slow = slow.next;

        fast = fast.next.next;
    }

    prev.next = newNode;

    newNode.next = slow;
}
```

15. Insert After Node

```
void insertAfter(Node prev, int val) {

    if(prev == null)
        return;

    Node temp = new Node(val);

    temp.next = prev.next;

    prev.next = temp;
}
```

16. Insert Before Node

```
void insertBefore(int target, int val) {

    if(head == null)
        return;

    if(head.val == target) {

        addFirst(val);
        return;
    }

    Node temp = head;
```

```
while(temp.next != null && temp.next.val != target) {  
  
    temp = temp.next;  
}  
  
if(temp.next != null) {  
  
    Node n = new Node(val);  
  
    n.next = temp.next;  
  
    temp.next = n;  
}  
}
```

DELETION OPERATIONS

17. Delete from Beginning

```
void deleteFirst() {  
  
    if(head == null)  
        return;  
  
    head = head.next;  
}
```

18. Delete from End

```
void deleteLast() {  
  
    if(head == null)  
        return;  
  
    if(head.next == null) {  
  
        head = null;  
        return;  
}
```

```
    }

    Node temp = head;

    while(temp.next.next != null) {

        temp = temp.next;
    }

    temp.next = null;
}
```

19. Delete from Position

```
void deleteAt(int pos) {

    if(pos == 1) {

        deleteFirst();
        return;
    }

    Node temp = head;

    for(int i = 1; i < pos - 1; i++) {

        temp = temp.next;
    }

    temp.next = temp.next.next;
}
```

20. Delete Middle Node

```
void deleteMiddle() {

    if(head == null || head.next == null) {

        head = null;
        return;
    }
}
```

```

    }

    Node slow = head;
    Node fast = head;
    Node prev = null;

    while(fast != null && fast.next != null) {

        prev = slow;

        slow = slow.next;

        fast = fast.next.next;
    }

    prev.next = slow.next;
}

```

21. Delete Entire Linked List

```

void clear() {

    head = null;

}

```

22. Delete Duplicate Nodes

```

void removeDuplicates() {

    Node current = head;

    while(current != null) {

        Node temp = current;

        while(temp.next != null) {

            if(temp.next.val == current.val)
                temp.next = temp.next.next;
            else

```

```
        temp = temp.next;
    }

    current = current.next;
}
}
```

ACCESS OPERATIONS

23. Get First Node

```
int getFirst() {
    return head.val;
}
```

24. Get Last Node

```
int getLast() {
    Node temp = head;

    while(temp.next != null)
        temp = temp.next;

    return temp.val;
}
```

25. Get Middle Node

```
int getMiddle() {
    Node slow = head;
    Node fast = head;

    while(fast != null && fast.next != null) {
```

```
        slow = slow.next;

        fast = fast.next.next;
    }

    return slow.val;
}
```

26. Get Node at Index

```
int getAt(int idx) {

    Node temp = head;

    for(int i = 0; i < idx; i++) {

        temp = temp.next;
    }

    return temp.val;
}
```

27. Get Kth Node from End

```
int kthFromEnd(int k) {

    Node slow = head;
    Node fast = head;

    for(int i = 0; i < k; i++) {

        fast = fast.next;
    }

    while(fast != null) {

        slow = slow.next;

        fast = fast.next;
    }
}
```

```
    }  
  
    return slow.val;  
}
```

REVERSE OPERATIONS

28. Reverse Linked List Iterative

```
void reverse() {  
  
    Node prev = null;  
    Node curr = head;  
  
    while(curr != null) {  
  
        Node next = curr.next;  
  
        curr.next = prev;  
  
        prev = curr;  
  
        curr = next;  
    }  
  
    head = prev;  
}
```

29. Reverse Linked List Recursive

```
Node reverseRec(Node head) {  
  
    if(head == null || head.next == null)  
        return head;  
  
    Node newHead = reverseRec(head.next);  
  
    head.next.next = head;
```

```
    head.next = null;

    return newHead;
}
```

30. Reverse in K Groups

```
Node reverseKGroup(Node head, int k) {

    Node curr = head;
    Node prev = null;
    Node next = null;

    int count = 0;

    while(curr != null && count < k) {

        next = curr.next;

        curr.next = prev;

        prev = curr;

        curr = next;

        count++;
    }

    if(next != null)
        head.next = reverseKGroup(next, k);

    return prev;
}
```

POINTER BASED OPERATIONS

31. Fast & Slow Pointer

```
void fastSlow() {  
  
    Node slow = head;  
    Node fast = head;  
  
    while(fast != null && fast.next != null) {  
  
        slow = slow.next;  
  
        fast = fast.next.next;  
    }  
  
    System.out.println(slow.val);  
}
```

32. Two Pointer Technique

```
void twoPointer() {  
  
    Node first = head;  
    Node second = head;  
  
    while(second != null && second.next != null) {  
  
        first = first.next;  
  
        second = second.next.next;  
    }  
}
```

33. Cycle Detection

```
boolean hasCycle() {
```

```
Node slow = head;
Node fast = head;

while(fast != null && fast.next != null) {

    slow = slow.next;

    fast = fast.next.next;

    if(slow == fast)
        return true;
}

return false;
}
```

34. Find Intersection Point

```
Node intersection(Node a, Node b) {

    java.util.HashSet<Node> set = new java.util.HashSet<>();

    while(a != null) {

        set.add(a);

        a = a.next;
    }

    while(b != null) {

        if(set.contains(b))
            return b;

        b = b.next;
    }

    return null;
}
```

35. Merge Linked Lists

```
Node merge(Node a, Node b) {  
  
    Node dummy = new Node(-1);  
  
    Node temp = dummy;  
  
    while(a != null && b != null) {  
  
        if(a.val < b.val) {  
  
            temp.next = a;  
  
            a = a.next;  
        }  
        else {  
  
            temp.next = b;  
  
            b = b.next;  
        }  
  
        temp = temp.next;  
    }  
  
    if(a != null)  
        temp.next = a;  
  
    if(b != null)  
        temp.next = b;  
  
    return dummy.next;  
}
```

SORTING & ARRANGEMENT OPERATIONS

36. Sort Linked List

```
void sort() {
```

```

Node current = head;

while(current != null) {

    Node index = current.next;

    while(index != null) {

        if(current.val > index.val) {

            int temp = current.val;

            current.val = index.val;

            index.val = temp;

        }

        index = index.next;

    }

    current = current.next;

}
}

```

37. Merge Sort in Linked List

```

Node mergeSort(Node head) {

    if(head == null || head.next == null)
        return head;

    Node mid = middleNode(head);

    Node nextMid = mid.next;

    mid.next = null;

    Node left = mergeSort(head);

    Node right = mergeSort(nextMid);

    return merge(left, right);

}

```

38. Odd Even Arrangement

```
Node oddEven(Node head) {  
  
    if(head == null)  
        return null;  
  
    Node odd = head;  
    Node even = head.next;  
    Node evenHead = even;  
  
    while(even != null && even.next != null) {  
  
        odd.next = even.next;  
  
        odd = odd.next;  
  
        even.next = odd.next;  
  
        even = even.next;  
    }  
  
    odd.next = evenHead;  
  
    return head;  
}
```

39. Remove Duplicates

```
void removeDuplicateSorted() {  
  
    Node temp = head;  
  
    while(temp != null && temp.next != null) {  
  
        if(temp.val == temp.next.val)  
            temp.next = temp.next.next;  
        else  
            temp = temp.next;  
    }  
}
```

```
}  
}
```

40. Partition Linked List

```
Node partition(Node head, int x) {  
  
    Node small = new Node(0);  
    Node greater = new Node(0);  
  
    Node s = small;  
    Node g = greater;  
  
    while(head != null) {  
  
        if(head.val < x) {  
  
            s.next = head;  
            s = s.next;  
        }  
        else {  
  
            g.next = head;  
            g = g.next;  
        }  
  
        head = head.next;  
    }  
  
    g.next = null;  
  
    s.next = greater.next;  
  
    return small.next;  
}
```

SPECIAL LINKED LIST PROBLEMS

41. Palindrome Linked List

```
boolean isPalindrome() {  
  
    java.util.ArrayList<Integer> list = new java.util.ArrayList<>();  
  
    Node temp = head;  
  
    while(temp != null) {  
  
        list.add(temp.val);  
  
        temp = temp.next;  
    }  
  
    int i = 0;  
    int j = list.size() - 1;  
  
    while(i < j) {  
  
        if(!list.get(i).equals(list.get(j)))  
            return false;  
  
        i++;  
        j--;  
    }  
  
    return true;  
}
```

42. Fold Linked List

```
void fold() {  
  
    reverse();  
}
```

43. Unfold Linked List

```
void unfold() {  
    reverse();  
}
```

44. Rotate Linked List

```
Node rotate(Node head, int k) {  
  
    if(head == null)  
        return null;  
  
    Node temp = head;  
  
    int len = 1;  
  
    while(temp.next != null) {  
        temp = temp.next;  
        len++;  
    }  
  
    temp.next = head;  
  
    k = k % len;  
  
    int move = len - k;  
  
    while(move-- > 0)  
        temp = temp.next;  
  
    head = temp.next;  
  
    temp.next = null;  
  
    return head;  
}
```

45. Clone Linked List

```
Node cloneList(Node head) {  
  
    if(head == null)  
        return null;  
  
    Node newHead = new Node(head.val);  
  
    Node temp1 = head.next;  
  
    Node temp2 = newHead;  
  
    while(temp1 != null) {  
  
        temp2.next = new Node(temp1.val);  
  
        temp1 = temp1.next;  
  
        temp2 = temp2.next;  
    }  
  
    return newHead;  
}
```

46. Flatten Linked List

```
Node flatten(Node root) {  
  
    return root;  
}
```

CIRCULAR LINKED LIST OPERATIONS

47. Circular Traversal

```
void circularPrint(Node head) {
```

```
if(head == null)
    return;

Node temp = head;

do {

    System.out.print(temp.val + " ");

    temp = temp.next;

} while(temp != head);
}
```

48. Insert in Circular Linked List

```
Node insertCircular(Node head, int val) {

    Node temp = new Node(val);

    if(head == null) {

        temp.next = temp;

        return temp;
    }

    Node curr = head;

    while(curr.next != head)
        curr = curr.next;

    curr.next = temp;

    temp.next = head;

    return head;
}
```

49. Delete in Circular Linked List

```
Node deleteCircular(Node head) {  
  
    if(head == null)  
        return null;  
  
    if(head.next == head)  
        return null;  
  
    Node temp = head;  
  
    while(temp.next.next != head)  
        temp = temp.next;  
  
    temp.next = head;  
  
    return head;  
}
```

DOUBLY LINKED LIST OPERATIONS

50. Doubly Node Class

```
class DNode {  
  
    int val;  
  
    DNode prev;  
    DNode next;  
  
    DNode(int val) {  
  
        this.val = val;  
    }  
}
```

51. Forward Traversal

```
void forward(DNode head) {  
  
    DNode temp = head;  
  
    while(temp != null) {  
  
        System.out.print(temp.val + " ");  
  
        temp = temp.next;  
    }  
}
```

52. Backward Traversal

```
void backward(DNode tail) {  
  
    DNode temp = tail;  
  
    while(temp != null) {  
  
        System.out.print(temp.val + " ");  
  
        temp = temp.prev;  
    }  
}
```

53. Insert at Beginning in DLL

```
DNode insertDLLFirst(DNode head, int val) {  
  
    DNode temp = new DNode(val);  
  
    temp.next = head;  
  
    if(head != null)  
        head.prev = temp;  
}
```

```
    head = temp;

    return head;
}
```

54. Insert at End in DLL

```
DNode insertDLLEnd(DNode head, int val) {

    DNode temp = new DNode(val);

    if(head == null)
        return temp;

    DNode curr = head;

    while(curr.next != null)
        curr = curr.next;

    curr.next = temp;

    temp.prev = curr;

    return head;
}
```

55. Delete Node in DLL

```
DNode deleteDLL(DNode head) {

    if(head == null)
        return null;

    head = head.next;

    if(head != null)
        head.prev = null;
}
```

```
    return head;
}
```

56. Reverse DLL

```
DNode reverseDLL(DNode head) {

    DNode temp = null;
    DNode curr = head;

    while(curr != null) {

        temp = curr.prev;

        curr.prev = curr.next;

        curr.next = temp;

        curr = curr.prev;
    }

    if(temp != null)
        head = temp.prev;

    return head;
}
```

ADVANCED OPERATIONS

57. Detect Loop

```
boolean detectLoop() {

    return hasCycle();
}
```

58. Remove Loop

```
void removeLoop() {  
  
    Node slow = head;  
    Node fast = head;  
  
    boolean loop = false;  
  
    while(fast != null && fast.next != null) {  
  
        slow = slow.next;  
  
        fast = fast.next.next;  
  
        if(slow == fast) {  
  
            loop = true;  
            break;  
        }  
    }  
  
    if(loop) {  
  
        slow = head;  
  
        while(slow.next != fast.next) {  
  
            slow = slow.next;  
  
            fast = fast.next;  
        }  
  
        fast.next = null;  
    }  
}
```

59. Find Loop Start

```
Node loopStart() {  
  
    Node slow = head;
```

```

Node fast = head;

while(fast != null && fast.next != null) {

    slow = slow.next;

    fast = fast.next.next;

    if(slow == fast)
        break;
}

slow = head;

while(slow != fast) {

    slow = slow.next;

    fast = fast.next;
}

return slow;
}

```

60. Add Two Linked Lists

```

Node addTwo(Node l1, Node l2) {

    Node dummy = new Node(0);

    Node temp = dummy;

    int carry = 0;

    while(l1 != null || l2 != null || carry != 0) {

        int sum = carry;

        if(l1 != null) {

            sum += l1.val;

            l1 = l1.next;
        }
    }
}

```

```

        if(l2 != null) {

            sum += l2.val;

            l2 = l2.next;
        }

        temp.next = new Node(sum % 10);

        carry = sum / 10;

        temp = temp.next;
    }

    return dummy.next;
}

```

61. Multiply Linked Lists

```

long multiply(Node a, Node b) {

    long n1 = 0;
    long n2 = 0;

    while(a != null) {

        n1 = n1 * 10 + a.val;

        a = a.next;
    }

    while(b != null) {

        n2 = n2 * 10 + b.val;

        b = b.next;
    }

    return n1 * n2;
}

```

62. Copy Random Pointer List

```
class RandomNode {  
    int val;  
  
    RandomNode next;  
    RandomNode random;  
  
    RandomNode(int val) {  
        this.val = val;  
    }  
}
```